

Obstacles for a Llama

Uma lama quer viajar pelo Planalto Andino. Ela tem um mapa do planalto na forma de uma matriz de $N \times M$ células quadradas. As linhas do mapa estão numeradas de 0 a $N - 1$, de cima para baixo, e as colunas estão numeradas de 0 a $M - 1$, da esquerda para a direita. A célula do mapa na linha i e na coluna j ($0 \leq i < N, 0 \leq j < M$) é designada por (i, j) .

A lama estudou o clima do planalto e descobriu que todas as células em cada linha do mapa têm a mesma **temperatura** e todas as células em cada coluna do mapa têm a mesma **humidade**. A lama deu-te dois vectors de números inteiros T e H de comprimento N e M respetivamente. Aqui, $T[i]$ ($0 \leq i < N$) indica a temperatura das células na linha i , e $H[j]$ ($0 \leq j < M$) indica a humidade das células na coluna j .

A lama estudou também a flora do planalto e verificou que uma célula (i, j) está **livre de vegetação** se e só se a sua temperatura for superior à sua humidade, formalmente $T[i] > H[j]$.

A lama só pode viajar através do planalto seguindo **caminhos válidos**. Um caminho válido é uma sequência de células distintas que satisfazem as seguintes condições:

- Cada par de células consecutivas no caminho partilha um lado comum.
- Todas as células do caminho estão livres de vegetação.

A tua tarefa é responder a Q perguntas. Para cada pergunta, são dados quatro números inteiros: L, R, S e D . Tens de determinar se existe um caminho válido tal que:

- O caminho começa na célula $(0, S)$ e termina na célula $(0, D)$.
- Todas as células do caminho encontram-se entre as colunas L a R , inclusive.

É garantido que tanto $(0, S)$ como $(0, D)$ estão livres de vegetação.

Detalhes de Implementação

A primeira função que deves implementar é:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : um vector de comprimento N especificando a temperatura em cada linha.
- H : um vector de comprimento M especificando a humidade em cada coluna.

- Esta função é chamada exatamente uma vez para cada caso de teste, antes de qualquer chamada a `can_reach`.

A segunda função que debes implementar é:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : inteiros que descrevem uma pergunta.
- Esta função é chamada Q vezes para cada caso de teste.

Esta função deve devolver `true` se e só se existir um caminho válido da célula $(0, S)$ para a célula $(0, D)$, de modo a que todas as células do caminho se encontrem entre as colunas L a R , inclusive.

Restrições

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ para cada i tal que $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ para cada j tal que $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Ambas as células $(0, S)$ e $(0, D)$ estão livres de vegetação.

Subtarefas

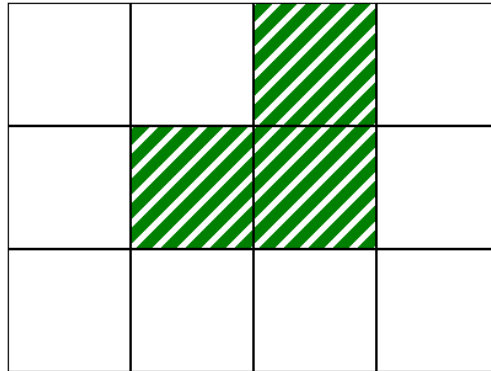
Subtarefa	Pontuação	Restrições adicionais
1	10	$L = 0, R = M - 1$ para cada pergunta. $N = 1$.
2	14	$L = 0, R = M - 1$ para cada pergunta. $T[i - 1] \leq T[i]$ para cada i tal que $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ para cada pergunta. $N = 3$ e $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ para cada pergunta. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ para cada pergunta.
6	17	Sem restrições adicionais.

Exemplo

Considera a seguinte chamada:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

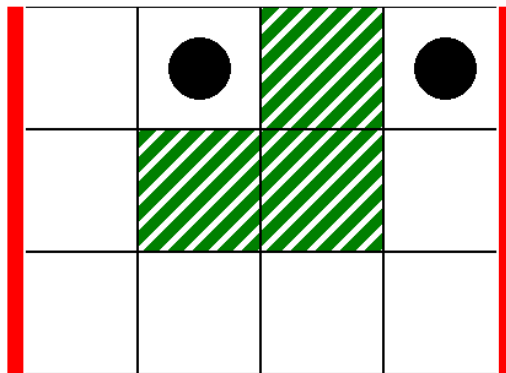
Isto corresponde ao mapa da imagem seguinte, onde as células brancas estão livres de vegetação:



Como primeira pergunta, considera a seguinte chamada:

```
can_reach(0, 3, 1, 3)
```

Isto corresponde ao cenário da imagem seguinte, em que as linhas verticais grossas indicam o intervalo de colunas de $L = 0$ a $R = 3$, e os discos pretos indicam as células inicial e final:



Neste caso, a lama pode ir da célula $(0, 1)$ à célula $(0, 3)$ através do seguinte caminho válido:

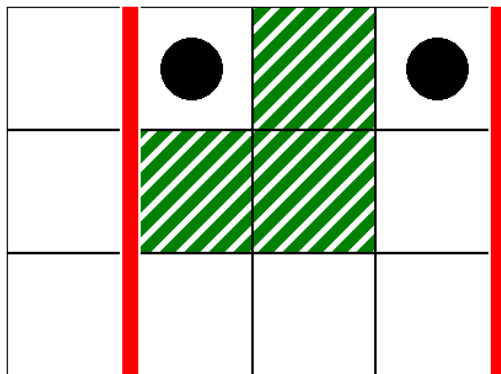
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Portanto, esta chamada deve devolver `true`.

Como segunda questão, considera a seguinte chamada:

```
can_reach(1, 3, 1, 3)
```

Isto corresponde ao cenário da imagem seguinte:



Neste caso, não existe um caminho válido da célula $(0, 1)$ para a célula $(0, 3)$, de modo a que todas as células do caminho se encontrem nas colunas 1 a 3, inclusive. Portanto, esta chamada deve devolver `false`.

Avaliador Exemplo

Formato do input:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Aqui, $L[k], R[k], S[k]$ e $D[k]$ ($0 \leq k < Q$) especificam os parâmetros para cada chamada a `can_reach`.

Formato do output:

```
A[0]
A[1]
...
A[Q-1]
```

Aqui, $A[k]$ ($0 \leq k < Q$) é 1 se a chamada `can_reach(L[k], R[k], S[k], D[k])` devolveu `true` e 0 caso contrário.